Deep Learning for Sequence Data

Recurrent Neural Networks (RNNs)

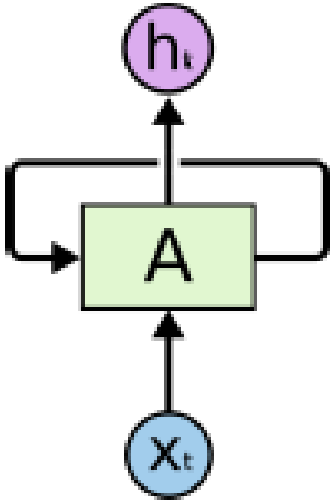
FRC-EVL

Motivation

- Humans process information sequentially and retain context.
- Traditional neural networks lack memory of previous inputs.
- Example: Understanding events in a movie requires past context.
- Need models that can handle sequential dependencies.

What is an RNN?

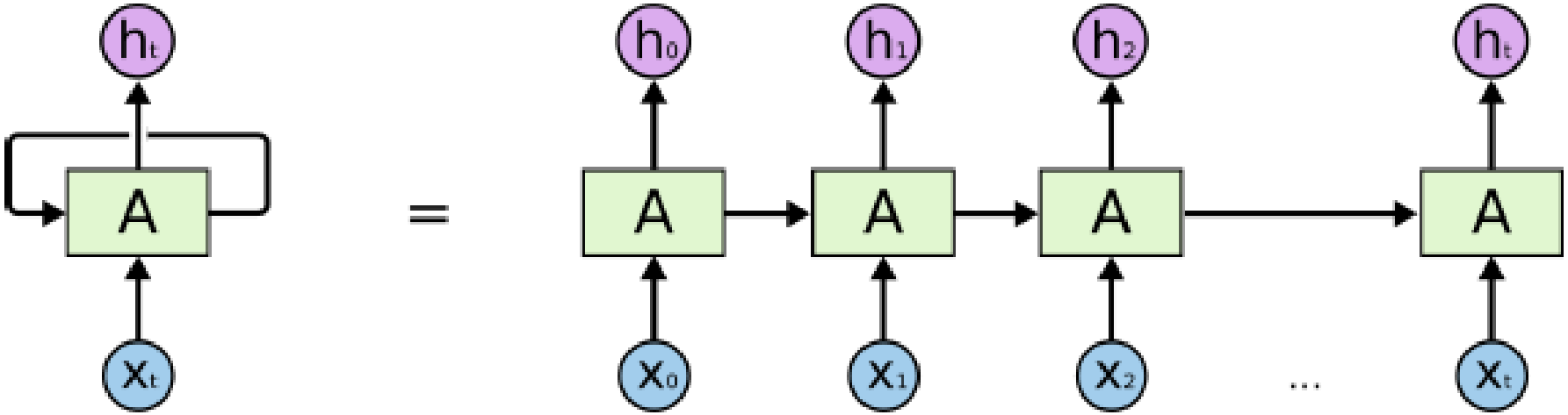
- Recurrent Neural Networks (RNNs) introduce loops.
- These loops allow information to persist across steps (throughout the sequence).
- Output at each step depends on:
  - Current input
  - Previous hidden state
- Suitable for sequential data.



RNN Unrolling

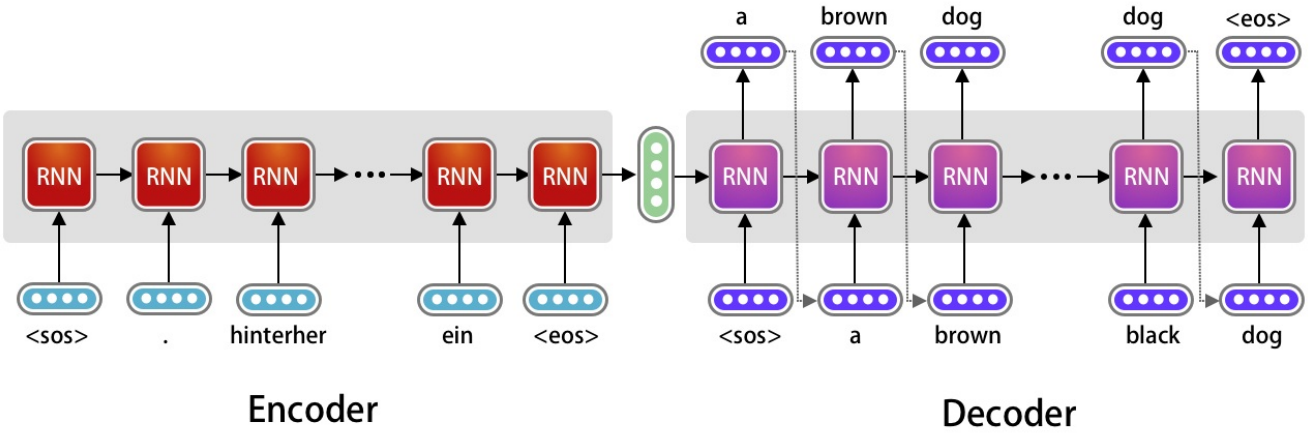
- RNN can be seen as repeated copies of the same network.
- Each copy processes one step.
- Information flows through hidden states.

Unrolled RNN structure shows sequence processing over tokens



Applications of RNNs

- Speech recognition
- Language modeling
- Machine translation
- Image captioning



How RNNs Work

- Process sequence element by element.
- Maintain a hidden state h_t.
- Hidden state summarizes past information.
- State is reset between independent sequences.

RNN Layer Structure

- Input sequence: $[x_1, x_2, ..., x_n]$
- At each step:
  - Input x_t
  - Previous state h_{t-1}
  - Output new state h_t
- Final output is h_n

RNN Computation

Hidden state update

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

- W: input weights
- U: recurrent weights
- b: bias
- Same parameters shared across all steps

Number of parameters in the RNN layer

$$\text{Params} = (units \times input) + (units \times units) + units$$

- Input weights
- Recurrent weights
- Bias vector

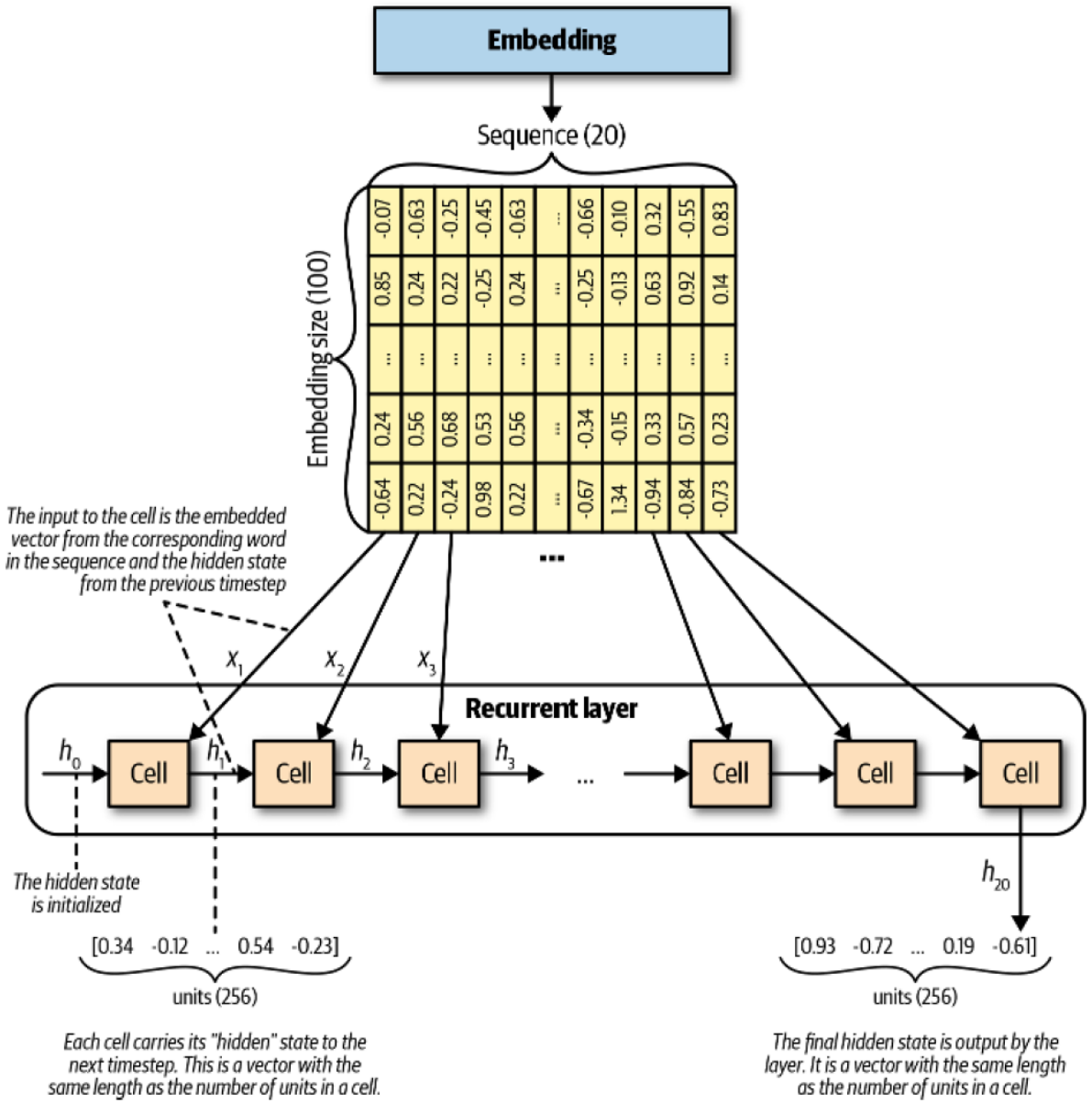
Why tanh in RNNs?

RNN Update

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

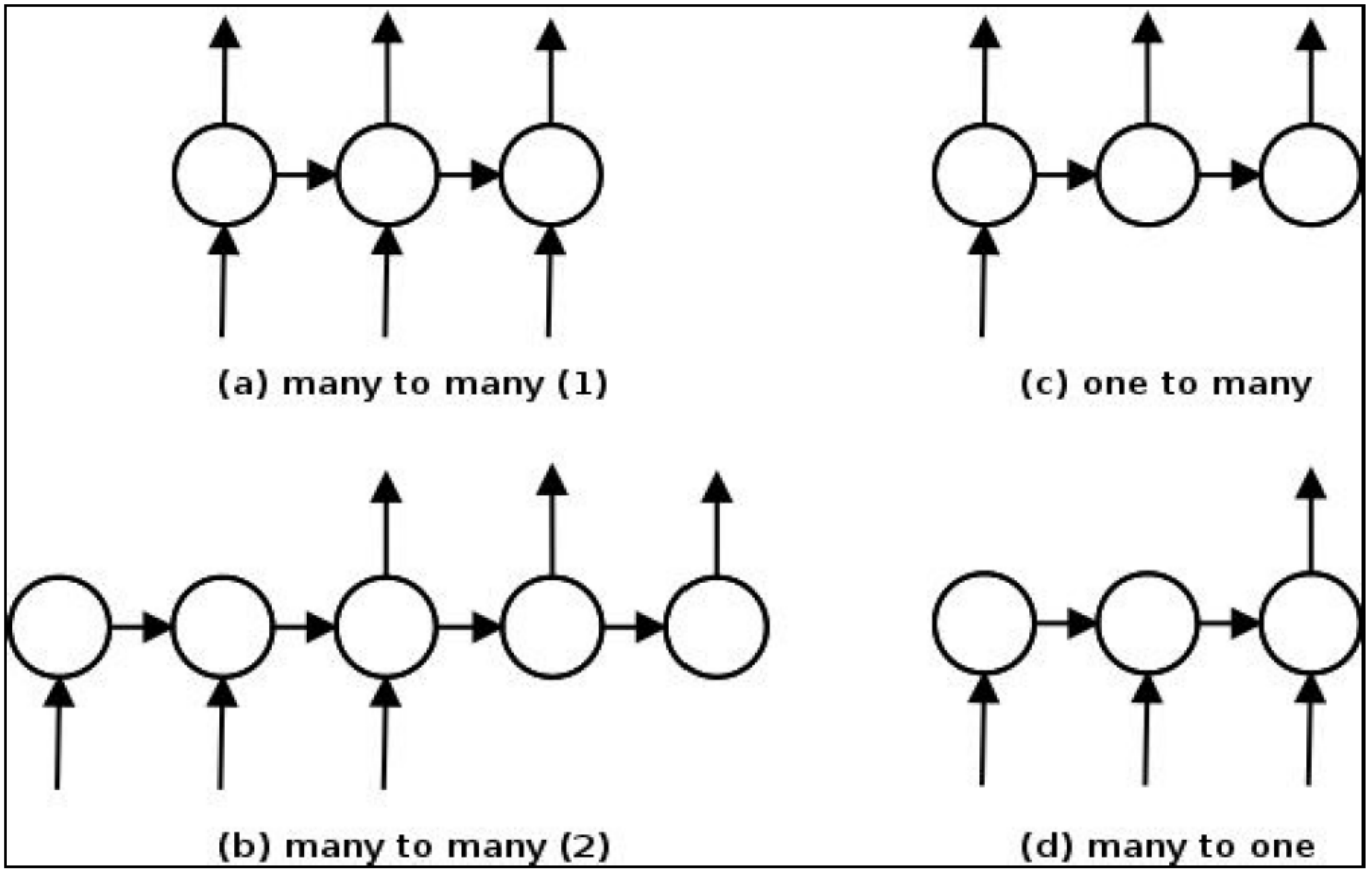
- **Non-linearity:** Enables learning complex temporal patterns.
- **Bounded output:** $\tanh(x) \in [-1, 1]$ prevents exploding values.
- **Zero-centered:** Improves optimization and gradient flow.
- **Memory control:**
  - Values near $\pm 1 \rightarrow$ strong signal (remember)
  - Values near $0 \rightarrow$ weak signal (forget)
- **Limitation:** Saturation can cause vanishing gradients.

How a single sequence flows through a recurrent layer



RNN Topologies

- Many-to-many (sequence labeling)
- Many-to-many (translation)
- One-to-many (image captioning)
- Many-to-one (sentiment analysis)



Conclusion

- RNNs are powerful for sequential data.
- Capture temporal dependencies via hidden states.
- Widely used in NLP and time series.
- RNNs are increasingly complemented or enhanced by attention-based models such as Transformers, and hybrid architectures (e.g., Temporal Fusion Transformer) often combine RNN/LSTM with attention mechanisms.